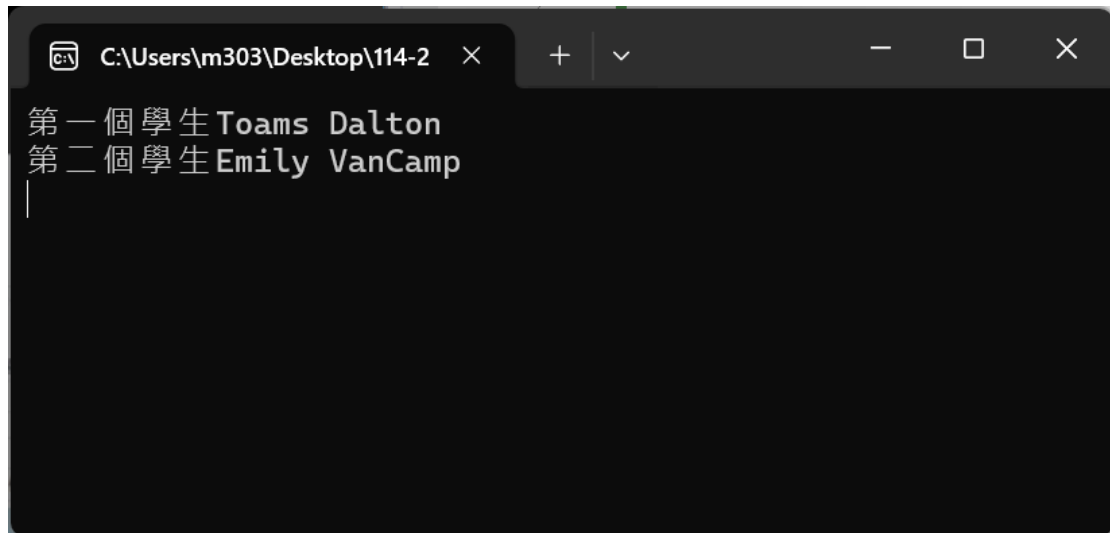




```
C:\Users\m303\Desktop\114-2 × + - □ ×
第一個學生Toams Dalton
第二個學生Emily VanCamp
|
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static System.Console; // 使用 static 匯入，讓 WriteLine、ReadKey 等可直接呼叫

namespace ConsoleApp1
{
    // internal：類別只在同一組件內可見
    internal class Program
    {
        // Main 為程式進入點（靜態方法）
        // 參數 args 為字串陣列，可接收命令列引數
        static void Main(string[] args)
        {

            // 建立兩個 Student 類別的實例（物件）
            // s1 與 s2 為參考型別變數，各自指向新建立的 Student 物件
            Student s1 = new Student();
            Student s2 = new Student();

            // 呼叫 Student 的 InputName 方法，將姓名字串設定到物件內部
            // 這裡使用範例英文姓名作為參數傳入
            s1.InputName("Toams Dalton");
            s2.InputName("Emily VanCamp");
        }
    }
}
```

```

        // 使用字串內插 (interpolation) 輸出學生姓名
        // 假設 Student.ShowName() 會回傳儲存在物件內的姓名字串
        WriteLine($"第一個學生{s1.ShowName()}");
        WriteLine($"第二個學生{s2.ShowName()}");

        // ReadKey(): 等待使用者按下任意鍵，避免主控台視窗立即關閉
        ReadKey();
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

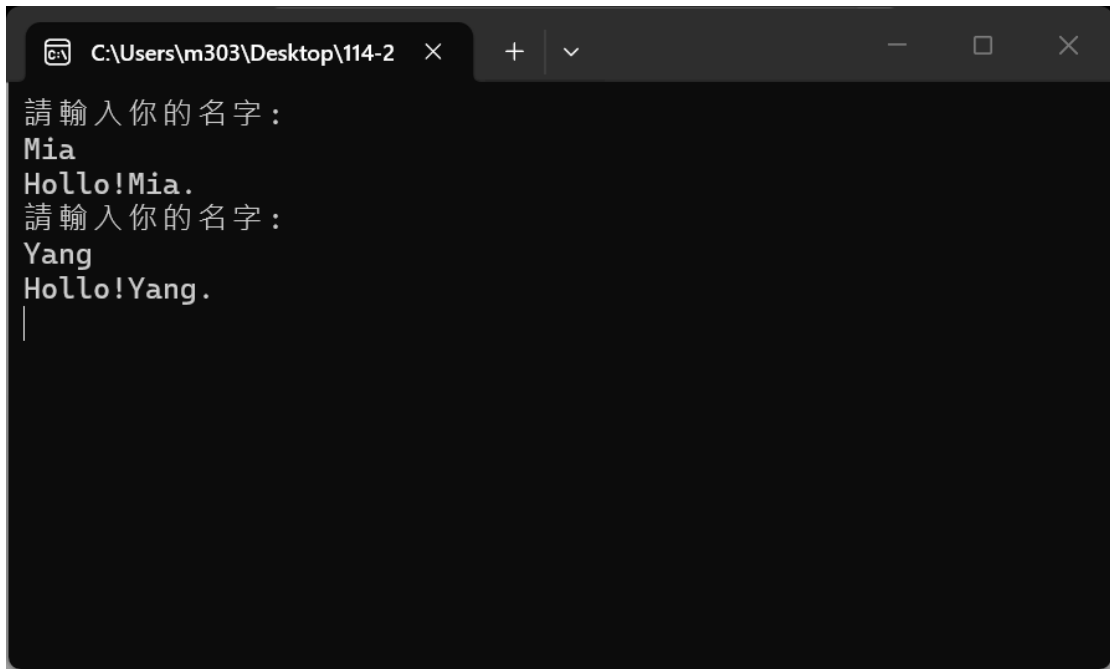
namespace ConsoleApp1
{
    // internal: 此類別僅在同一組件內可見，外部組件無法直接存取
    internal class Student
    {
        // 私有欄位，用來儲存學生的姓名
        // private 表示此欄位只能在 Student 類別內部存取，外部無法直接讀取或修改
        private string name;

        // InputName: 設定學生姓名的方法
        // 參數 title 為欲儲存的姓名字串，方法以 expression-bodied member 的形式將參數
        // 指派給私有欄位 name
        public void InputName(string title) => name = title;

        // ShowName: 回傳學生姓名的方法
        // 若尚未呼叫 InputName 設定姓名，回傳值可能為 null，呼叫端應視情況處理或先行檢
        // 查
        public string ShowName() => name;
    }
}

```

```
}  
}
```



```
C:\Users\m303\Desktop\114-2 × + ▾  
請輸入你的名字：  
Mia  
Hollo!Mia.  
請輸入你的名字：  
Yang  
Hollo!Yang.  
|
```

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
using static System.Console;  
  
namespace ConsoleApp2  
{  
    internal class Program  
    {  
        // 主程式類別  
        static void Main(string[] args)  
        {  
            // 建立第一個 Person 實例，要求使用者輸入名字並顯示問候  
            Person p1 = new Person();  
            WriteLine("請輸入你的名字:");  
            p1.Name = ReadLine();  
            p1.Display();  
  
            // 建立第二個 Person 實例，重複輸入與顯示流程
```

```

        Person p2 = new Person();
        WriteLine("請輸入你的名字:");
        p2.Name = ReadLine();
        p2.Display();

        // 等待使用者按任意鍵後結束程式
        ReadKey();

    }
}

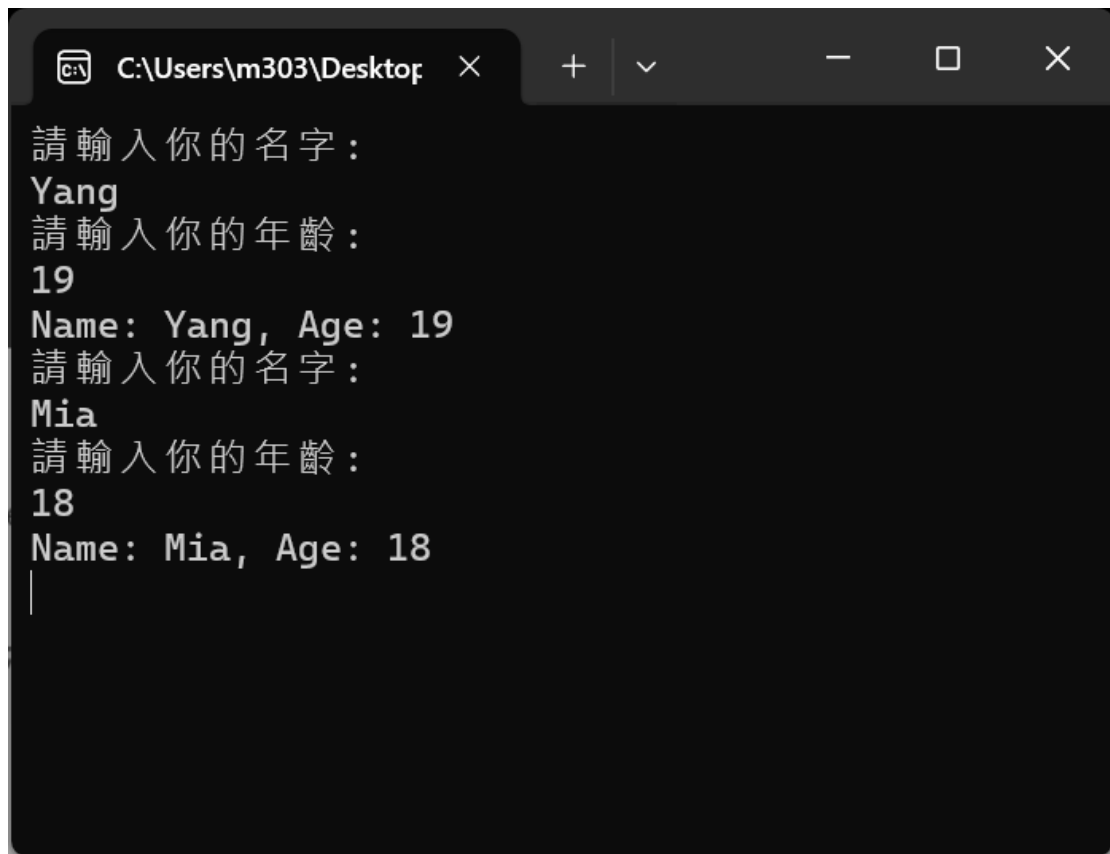
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp2
{
    // 表示一個人的類別，包含姓名欄位與相關操作
    internal class Person
    {
        // 儲存姓名的私有欄位
        private string name;

        // 公開的姓名屬性，提供取得與設定功能
        public string Name
        {
            get { return name; }
            set { name = value; }
        }

        // 將問候訊息輸出到主控台
        public void Display()=>Console.WriteLine($"Hollo!{Name}.");
    }
}

```



```
請輸入你的名字：
Yang
請輸入你的年齡：
19
Name: Yang, Age: 19
請輸入你的名字：
Mia
請輸入你的年齡：
18
Name: Mia, Age: 18
|
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static System.Console;

namespace ConsoleApp3
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // 建立第一個 Student 物件
            Student s1 = new Student();

            // 提示使用者輸入名字，並將輸入值指定給 s1 的 Name 屬性
            WriteLine("請輸入你的名字:");
            s1.Name = ReadLine();
        }
    }
}
```

```

        // 提示使用者輸入年齡，並將輸入的字串解析為整數指定給 s1 的 Age 屬性
        WriteLine("請輸入你的年齡:");
        s1.Age = int.Parse(ReadLine());

        // 顯示 s1 的資訊
        s1.ShowMessage();

        // 建立第二個 Student 物件
        Student s2 = new Student();

        // 提示使用者輸入名字，並將輸入值指定給 s2 的 Name 屬性
        WriteLine("請輸入你的名字:");
        s2.Name = ReadLine();

        // 提示使用者輸入年齡，並將輸入的字串解析為整數指定給 s2 的 Age 屬性
        WriteLine("請輸入你的年齡:");
        s2.Age = int.Parse(ReadLine());

        // 顯示 s2 的資訊
        s2.ShowMessage();

        // 等待使用者按任意鍵後結束程式
        ReadKey();
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp3
{
    internal class Student
    {
        // Student 類別：代表一個學生的資料結構，包含姓名與年齡屬性以及顯示資訊的方法

```

```

        public string Name { get; set; }

        // Age 屬性：以整數表示學生的年齡
        public int Age { get; set; }

        public void ShowMessage()
        {
            // 將學生的姓名與年齡輸出到主控台
            Console.WriteLine($"Name: {Name}, Age: {Age}");
        }
    }
}

```

員工姓名：珍妮佛  
實領薪水：30000

---

經理姓名：艾莉絲  
實領薪水：40000  
實領獎金：20000  
合計薪資：60000

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WindowsFormsApp1

```

```

{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        private void LblMsg_Click(object sender, EventArgs e)
        {

        }

        private void Form1_Load(object sender, EventArgs e)
        {
            Empolyee Jennifer = new Empolyee(); // Employee 父類別
            Jennifer.Name = "珍妮佛";
            Jennifer.Salary = 30000;
            LblMsg.Text = $"員工姓名：{Jennifer.Name} \n 實領薪水：{Jennifer.Salary}";
            LblMsg.Text += "\n===== \n";
            Manager Alice = new Manager(); // Manager 子類別
            Alice.Name = "艾莉絲";
            Alice.Salary = 40000;
            Alice.Bonus = 20000; // Manager 子類別新增的 Bonus 屬性
            LblMsg.Text += Alice.GetTotal(); //Manager 子類別新增的 GetTotal()方法
        }
    }
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WindowsFormsApp1
{
    public class Empolyee

```



```

{
    public string Name { get; set; }
    public int _salary;
    public int Salary
    {
        get
        {
            return _salary;
        }
        set
        {
            if (value <= 20000) value = 20000;
            if (value >= 40000) value = 40000;
            _salary = value;
        }
    }
}

```

```

public class Manager : Employee
{
    private int _bonus;
    public int Bonus
    {
        get
        {
            return _bonus;
        }
        set
        {
            if (value <= 10000) value = 10000;
            if (value >= 50000) value = 50000;
            _bonus = value;
        }
    }
}

public string GetTotal()
{

```

```
        return $"經理姓名：{Name}\n 實領薪水：{Salary}\n 實領獎金：{Bonus}\n 合計薪  
資：{ Bonus + Salary}";  
    }  
}  
}
```